

1 - La récursivité

Une fonction peut s'appeler elle-même. La condition d'arrêt s'écrit avec si ... sinon, qui produit ici une valeur plutôt qu'un texte :

Les premières factorielles : 1, 2, 6, 24, 120, 720.

La suite de Fibonacci se définit de la même façon, avec deux appels par tour :

Les douze premiers termes, dressés par une boucle qui appelle la fonction à chaque rang :

Rang	Terme
0	0
1	1
2	1
3	2
4	3
5	5
6	8
7	13
8	21
9	34
10	55
11	89

La récursion est bornée : au-delà de deux cents appels imbriqués, docd g s'arrête et le dit, plutôt que d'épuiser la pile.

2 - Les algorithmes classiques

Les exercices du programme s'écrivent avec ce qui précède : une fonction, une boucle, une collection reçue en copie.

2.1 - Chercher

Le `retourne` placé dans la boucle en sort sur-le-champ : c'est le geste que montre tout manuel. Dans une série triée, la dichotomie coupe l'intervalle en deux à chaque tour.

11 occupe la position 4 ; recherche(premiers ; 4) ne le trouve pas : -1.

2.2 - Trier

Méthode	Résultat
Insertion	{1 ; 2 ; 5 ; 5 ; 6 ; 9}
Sélection	{1 ; 2 ; 5 ; 5 ; 6 ; 9}
Série d'origine	{5 ; 2 ; 9 ; 1 ; 5 ; 6}

La série de départ n'a pas bougé : une fonction rend une valeur, elle ne modifie rien à distance. La primitive `tri(désordre)` donne le même résultat — {1 ; 2 ; 5 ; 5 ; 6 ; 9} — mais on n'apprend pas à trier en l'appelant.

2.3 - Fusionner deux listes triées

{1 ; 2 ; 3 ; 4 ; 7 ; 8 ; 9}

À la sortie de la boucle, l'une des deux tranches finales est toujours vide — une tranche dont la borne de gauche dépasse celle de droite n'est pas une faute. C'est ce qui permet de conclure en une ligne.

3 - Les structures de données

3.1 - Les écrire à la main

Sept lignes suffisent, et aucune n'est du moteur : la structure du programme de terminale se démontre au tableau au lieu d'être cachée dans le logiciel. On les nomme ici avec un suffixe, car une fonction qu'on écrit masque la primitive du même nom — c'est voulu, pour que l'élève puisse écrire la sienne, mais cela empêcherait d'employer les deux dans un même document.

La pile {1 ; 2 ; 3} a pour sommet 3 ; après dépilement il reste {1 ; 2}. La file {1 ; 2 ; 3} a pour tête 1 et laisse {2 ; 3}.

Le type de `p` n'est pas écrit : une valeur composée se pose telle quelle, `docdg` reconnaît ce qu'elle est.

3.2 - Celles du langage

Une fois l'exercice fait, le langage les fournit — avec leur discipline, qui est tout leur intérêt. Une file n'a pas de sommet, une pile n'a pas de tête, et aucune des deux ne s'indexe.

La pile {1 ; 2} a pour sommet 2 ; dépilée, il reste {1}. Elle est vide ? faux

La file {1 ; 2 ; 3} a pour tête 1 ; défilée, il reste {2 ; 3}.

3.3 - Rendre plusieurs valeurs : le p-uplet

Un p-uplet a une longueur fixe et des types qui peuvent différer, là où une collection est homogène et de longueur libre. Il s'écrit entre parenthèses, comme un point du plan.

$17 = 5 \times 3 + 2$ — la déliaison pose les deux noms d'un coup.

L'exercice qui justifie à lui seul le p-uplet : deux résultats, un seul parcours.

La plus petite valeur est 9, la plus grande 18.

Les membres se lisent aussi par leur rang, et les types peuvent différer :

(3 ; trois) — dont le second membre est trois.

4 - Tirer au hasard

aléatoire(a ; b) rend un entier entre les deux bornes, comprises. Deux compilations donnent deux tirages : c'est ce qu'une simulation attend.

Sur 60 lancers d'un dé, 9 six — pour une espérance de 10.

5 - Les objets

Une classe rassemble des données et ce qu'on sait en faire. Elle se déclare comme une phrase de mathématiques, et le type de ses attributs n'a pas besoin d'article : à ce niveau la prose n'apporte rien.

Le point Point(abscisse: 3 ; ordonnée: 4) a pour distance à l'origine 5. Translaté d'une unité dans chaque direction, il devient Point(abscisse: 4 ; ordonnée: 5).

Le nom d'une classe commence par une majuscule : c'est ce qui le distingue d'un type du langage. Un attribut se lit avec un point, une méthode s'appelle de même. Dans le corps d'une méthode, les attributs sont visibles par leur nom, sans rien devant : la formule s'écrit comme au tableau.

Un attribut se modifie sans soit, puisque rien n'est déclaré :

Le point q vaut Point(abscisse: 6 ; ordonnée: 2).

5.1 - Premier pilier : l'encapsulation

Une classe peut garder pour elle ce qui ne regarde pas le dehors. Un seul mot le dit, et tout ce qui n'est pas marqué reste visible.

Le titulaire se lit du dehors : Léa. Les intérêts se calculent : 5 — la méthode publique lit l'attribut privé et appelle la méthode privée, ce qui est tout l'objet de l'encapsulation.

Mais le solde ne se lit pas du dehors : ⚠ solde est un attribut privé de Compte : il ne se lit que depuis la classe

Ni la méthode privée ne s'appelle : ⚠ taux est une méthode privée de Compte : elle ne s'appelle que depuis la classe

5.2 - Deuxième pilier : l'héritage

Une classe peut reprendre une autre et la prolonger. Elle en reçoit les attributs, les méthodes et les secrets.

Rex reçoit de sa classe mère la méthode carte, qui donne Rex, et redéfinit cri, qui donne ouaf. L'animal, lui, garde le sien :

...

5.3 - Troisième pilier : le polymorphisme

Un chien tient la place d'un animal. C'est la valeur qui décide de la méthode appelée, non le type écrit :

ouaf — la fonction attend un animal, elle a reçu un chien, et c'est le cri du chien qui sort.

Rangés ensemble, chacun garde le sien :

[ouaf]
[miaou]

5.4 - Quatrième pilier : l'abstraction

Une classe peut dire ce que ses filles doivent savoir faire, sans dire comment. Une méthode déclarée sans membre droit n'a pas de corps.

Un carré de 3 a pour aire 9 ; un rectangle 2 sur 5 a pour aire 10. La fonction mesure ne sait rien de l'une ni de l'autre : elle sait seulement qu'une forme a une aire.

Une classe abstraite ne s'instancie pas — c'est le rôle de ses filles : ⚠ Forme est une classe abstraite : elle ne s'instancie pas, seules ses classes filles le font

6 - L'arbre binaire, écrit avec des objets

Les quatre piliers suffisent à écrire un arbre, sans avoir besoin d'une valeur nulle : l'arbre vide est une classe. La hiérarchie dit tout — un arbre est vide, ou c'est un nœud.

Cet arbre compte 3 nœuds et sa hauteur vaut 2.

Un attribut peut donc contenir un objet : c'est ce qui permet à une structure de se contenir elle-même.

7 - Les graphes

Un graphe se donne par sa liste d'adjacence : un dictionnaire dont chaque clé est un sommet et chaque valeur la liste de ses voisins.

Le graphe compte 4 sommets ; les voisins de A sont {B ; C}.

Les deux parcours diffèrent par une seule chose : la file donne le parcours en largeur, la pile le parcours en profondeur.

Depuis A, en largeur : ABCD. En profondeur : ACDB.

8 - Écrire un résultat sur une ligne

Les nombres premiers jusqu'à 50 : 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47.

La primitive jonction évite l'accumulateur, qui laisse toujours un séparateur de trop à la fin.

9 - Pour la suite

Les quatre piliers de la programmation orientée objet, les structures de données du programme et leurs parcours sont couverts. Ce qui reste tient moins du langage que de son emploi : écrire ses propres classes, ses propres structures, et les faire tenir dans un document qu'on donne à lire.